Creating the Database and Tables:

```sql
CREATE DATABASE IF NOT EXISTS ecommerce_db;
USE ecommerce_db;

CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    city VARCHAR(50),
    signup_date DATE
);

CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50),
    price DECIMAL(10,2),
    stock INT
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE,
    status VARCHAR(20),
    FOREIGN KEY (customer_id)
        REFERENCES Customers(customer_id)
);

CREATE TABLE Order_Items (
    order_item_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    purchase_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id)
        REFERENCES Orders(order_id),
    FOREIGN KEY (product_id)
        REFERENCES Products(product_id)
);
```

Inserted Values into Tables

customer_id	customer_name	email	city	signup_date
1	Aarav Sharma	aarav@example.com	Chennai	01/10/2025
2	Diya Iyer	diya@example.com	Bengaluru	02/15/2025
3	Rohan Mehta	rohan@example.com	Mumbai	03/20/2025
4	Ananya Rao	ananya@example.com	Chennai	04/05/2025
5	Karthik Nair	karthik@example.com	Kochi	05/12/2025
6	Meera Krishnan	meera@example.com	Hyderabad	06/18/2025
7	Arjun Patel	arjun@example.com	Mumbai	07/22/2025
8	Sneha Das	sneha@example.com	Kolkata	08/09/2025
9	Vikram Singh	vikram@example.com	Delhi	09/14/2025
10	Priya Menon	priya@example.com	Bengaluru	10/25/2025

product_id	product_name	category	price	stock
101	Laptop	Electronics	65000.00	12
102	Smartphone	Electronics	30000.00	25
103	Headphones	Accessories	2500.00	40
104	Wireless Mouse	Accessories	1200.00	50
105	Keyboard	Accessories	1800.00	30
106	Monitor	Electronics	15000.00	10
107	Smartwatch	Wearables	8000.00	18
108	Tablet	Electronics	22000.00	8
109	Power Bank	Accessories	1500.00	35
110	Bluetooth Speaker	Audio	3500.00	20

order_id	customer_id	order_date	status
1001	1	01/05/2026	Delivered
1002	2	01/08/2026	Delivered
1003	3	01/12/2026	Shipped
1004	1	01/15/2026	Delivered
1005	4	01/20/2026	Processing
1006	5	02/02/2026	Delivered
1007	6	02/10/2026	Shipped
1008	7	02/14/2026	Processing
1009	8	02/18/2026	Delivered
1010	10	02/25/2026	Processing

order_item_id	order_id	product_id	quantity	purchase_price
1	1001	101	1	65000.00
2	1001	103	2	2500.00
3	1002	102	1	30000.00
4	1002	104	1	1200.00
5	1003	106	2	15000.00
6	1003	105	1	1800.00
7	1004	107	1	8000.00
8	1004	109	2	1500.00
9	1005	108	1	22000.00
10	1006	103	3	2500.00
11	1006	104	2	1200.00
12	1007	102	1	30000.00
13	1008	110	2	3500.00
14	1009	105	2	1800.00
15	1010	101	1	65000.00

SELECT Query - Find all products priced above ₹5,000, sorted from highest to lowest price.

```
5   • SELECT product_name, category, price
6     FROM Products
7     WHERE price > 5000
8     ORDER BY price DESC;
9
```

Output | Result #1

product_name	category	price
Laptop	Electronics	65000.00
Smartphone	Electronics	30000.00
Tablet	Electronics	22000.00
Monitor	Electronics	15000.00
Smartwatch	Wearables	8000.00

SELECT Query - Multiple conditions with AND, OR, and LIKE

```sql
SELECT customer_id, customer_name, city
FROM Customers
WHERE (city = 'Chennai' OR city = 'Bengaluru')
  AND customer_name LIKE 'A%'
ORDER BY customer_name;
```

Output | Result #1

customer_id	customer_name	city
1	Aarav Sharma	Chennai
4	Ananya Rao	Chennai

INNER JOIN

```sql
SELECT
    o.order_id,
    c.customer_name,
    o.order_date,
    o.status
FROM Orders o
INNER JOIN Customers c
    ON o.customer_id = c.customer_id
ORDER BY o.order_date;
```

Output | Result #1

order_id	customer_name	order_date	status
1001	Aarav Sharma	01/05/2026	Delivered
1002	Diya Iyer	01/08/2026	Delivered
1003	Rohan Mehta	01/12/2026	Shipped
1004	Aarav Sharma	01/15/2026	Delivered
1005	Ananya Rao	01/20/2026	Processing
1006	Karthik Nair	02/02/2026	Delivered
1007	Meera Krishnan	02/10/2026	Shipped
1008	Arjun Patel	02/14/2026	Processing
1009	Sneha Das	02/18/2026	Delivered
1010	Priya Menon	02/25/2026	Processing

Multiple-table JOIN

```sql
SELECT
    o.order_id,
    c.customer_name,
    p.product_name,
    oi.quantity,
    oi.purchase_price,
    oi.quantity * oi.purchase_price AS line_total
FROM Orders o
INNER JOIN Customers c
    ON o.customer_id = c.customer_id
INNER JOIN Order_Items oi
    ON o.order_id = oi.order_id
INNER JOIN Products p
    ON oi.product_id = p.product_id
ORDER BY o.order_id, p.product_name;
```

Output | Result #1

order_id	customer_name	order_date	status
1001	Aarav Sharma	01/05/2026	Delivered
1002	Diya Iyer	01/08/2026	Delivered
1003	Rohan Mehta	01/12/2026	Shipped
1004	Aarav Sharma	01/15/2026	Delivered
1005	Ananya Rao	01/20/2026	Processing
1006	Karthik Nair	02/02/2026	Delivered
1007	Meera Krishnan	02/10/2026	Shipped
1008	Arjun Patel	02/14/2026	Processing
1009	Sneha Das	02/18/2026	Delivered
1010	Priya Menon	02/25/2026	Processing

Aggregation with GROUP BY and HAVING

```sql
SELECT
    p.product_name,
    SUM(oi.quantity) AS units_sold,
    SUM(oi.quantity * oi.purchase_price) AS total_sales
FROM Products p
INNER JOIN Order_Items oi
    ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
HAVING SUM(oi.quantity * oi.purchase_price) > 10000
ORDER BY total_sales DESC;
```

Output | Result #1

product_name	units_sold	total_sales
Laptop	2	130000.00
Smartphone	2	60000.00
Monitor	2	30000.00
Tablet	1	22000.00
Headphones	5	12500.00

Nested query

```sql
SELECT product_name, category, price
FROM Products
WHERE price > (
    SELECT AVG(price)
    FROM Products
)
ORDER BY price DESC;
```

Output | Result #1

product_name	category	price
Laptop	Electronics	65000.00
Smartphone	Electronics	30000.00
Tablet	Electronics	22000.00

Nested query with GROUP BY and HAVING

```sql
SELECT
    c.customer_id,
    c.customer_name,
    (
        SELECT SUM(oi.quantity * oi.purchase_price)
        FROM Orders o
        INNER JOIN Order_Items oi
            ON o.order_id = oi.order_id
        WHERE o.customer_id = c.customer_id
    ) AS total_spending
FROM Customers c
WHERE c.customer_id IN (
    SELECT o.customer_id
    FROM Orders o
    INNER JOIN Order_Items oi
        ON o.order_id = oi.order_id
    GROUP BY o.customer_id
    HAVING SUM(oi.quantity * oi.purchase_price) > 30000
)
ORDER BY total_spending DESC;
```

Output | Result #1

customer_id	customer_name	total_spending
1	Aarav Sharma	81000.00
10	Priya Menon	65000.00
3	Rohan Mehta	31800.00
2	Diya Iyer	31200.00